

DEPTH CHARGE

Personal Implementation Guide

Muhammad Anas (25K-0899) — Group Leader

*Game loop • Keyboard & submarine • Frame timing • Torpedoes • Collisions • Pause •
Project setup & integration*

Your folder / files	src/game.asm, src/shared/, src/anas/ (+ the GitHub repo itself)
Your working dates	Week 0 – Week 6: 5 Oct – 22 Nov 2026 (demo Mon 23 Nov)
GitHub repo	github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection
Group Leader	Muhammad Anas (25K-0899)
Member 2	Syed Alyan Hussain (25K-0648)
Member 3	Ali Raza (25K-0827)
Instructor / Demo	Muhammad Owais / Monday 23 November 2026

Read this first

- The code in this guide is a **worked example**. Type it yourself, change it if you want, and make sure you can explain **every line**. The viva is individual and covers the **whole** game, not only your part.
- Course rule: code taken from AI tools must be **disclosed** in the final report, e.g. "Reference examples from an AI-generated implementation guide were studied and adapted."
- There are three guides (one per member). They use the same files, names and dates, so they fit together.

1. Your Job in One Page

You are the **Group Leader**. You build the **heart of the game**: the loop that runs every frame, the keyboard, the submarine, the torpedoes and the hit checking. You also create the project on day one and join everybody's code together at the end. You have the most steps, but each one is small.

In simple words

- **Alyan** makes things *appear* on the screen. **Ali** makes the enemies, the score and the menus.
- **You** make the game *run*: read keys, move the sub, fire, check hits, and keep the speed steady.
- If your loop calls the right procedures in the right order, the game works.

1.1 Your steps

Step	Task	File	Week · dates
A1	Create the GitHub repo structure, shared files and stub files	whole repo	W0 · 5 – 7 Oct
A2	Main program and game loop (state machine)	src/game.asm	W1 · 12 – 15 Oct
A3	Frame timing (steady 80 ms per frame)	src/game.asm	W1 · 16 – 18 Oct
A4	Keyboard input and submarine movement	src/anas/input.inc	W2 · 19 – 25 Oct
A5	Torpedoes	src/anas/torpedo.inc	W3 · 26 Oct – 1 Nov
A6	Collision detection	src/anas/collision.inc	W4 · 2 – 8 Nov
A7	New-game reset and pause	src/game.asm, input.inc	W5 · 9 – 15 Nov
A8	Integration, testing and final report (leader)	all files	W6 · 16 – 22 Nov

1.2 What you give to the others / what you use from them

These are the "promises" between members. If your procedure takes and returns exactly what is written here, the three parts will fit together on the first try.

You provide	Input (registers)	Result	Used by
<code>HandleInput</code>	—	Reads all waiting keys, moves sub, calls <code>FireTorpedo</code> / <code>StartSonar</code>	GameLoop
<code>CheckCollisions</code>	—	Finds hits, calls <code>AddScore</code> , <code>LoseLife</code> , <code>StartExplosion</code>	GameLoop
<code>subX</code> , <code>Torpedoes</code> , <code>gameState</code>	(variables)	Sub column, torpedo array, current state	Alyan draws them

You use	Owner	What it does for you
<code>BuildFrame</code> , <code>Render</code>	Alyan	Draw the frame on screen
<code>StartExplosion</code>	Alyan	AL = column, AH = row → starts an explosion
<code>StartSonar</code> , <code>UpdateSonar</code>	Alyan	Turn sonar on / end it after 2 s
<code>InitSystem</code> , <code>InvalidateScreen</code>	Alyan	Console setup / force a full redraw
<code>ShowMenu</code> , <code>GameOverScreen</code>	Ali	Menu (sets <code>gameState</code>) and game-over screen
<code>SpawnCharge</code> , <code>UpdateCharges</code>	Ali	Create and move depth charges

You use	Owner	What it does for you
AddScore, LoseLife, ApplyLevel	Ali	EAX = points / take a life / set level speed
LoadHighScore, PlayFireSound, ...	Ali	High score file and sound effects

2. Your Weekly Timeline (5 Oct – 23 Nov)

We have **7 weeks** until the demo on **Monday 23 November**: one preparation week (Week 0) and six working weeks. Every week runs **Monday → Sunday**. Your deadline is **Sunday 9:00 pm**: by then the week's tasks must be finished, tested and sent to GitHub as a **Pull Request** (section 5).

Week	By Sunday 9 pm you must finish...	Pull Request title
Week 0 Mon 5 – Sun 11 Oct	A1 Create the repo structure from the starter folder and push it by Wed 7 Oct . Make sure Alyan and Ali have forked and can build by Fri 9 Oct . Merge their practice PRs. Study section 4.	Starter structure — pushed straight to <code>main</code> (the only time)
Week 1 Mon 12 – Sun 18 Oct	A2 <code>main</code> + <code>GameLoop</code> (+ a short <code>InitGame</code>) and A3 <code>FrameDelay</code> . Submit the proposal on Thu 15 Oct .	Week 1 – Anas – A2 A3
Week 2 Mon 19 – Sun 25 Oct	A4 <code>HandleInput</code> + submarine movement. Sunday night: merge all PRs, build, play → M1 .	Week 2 – Anas – A4
Week 3 Mon 26 Oct – Sun 1 Nov	A5 <code>FireTorpedo</code> + <code>UpdateTorpedoes</code> .	Week 3 – Anas – A5
Week 4 Mon 2 – Sun 8 Nov	A6 <code>CheckCollisions</code> . Sunday night: merge all PRs → M2 (core game playable).	Week 4 – Anas – A6
Week 5 Mon 9 – Sun 15 Nov	A7 full <code>InitGame</code> reset + pause. Start running the full test checklist.	Week 5 – Anas – A7
Week 6 Mon 16 – Sun 22 Nov	Wed 18 Nov — M3 feature freeze : merge everything, remove the stub files. Thu–Sat: bug fixes, full test checklist on a lab PC, put the final report together. Sun 22 Nov : tag <code>v1.0</code> → M4 .	Week 6 – Anas – A8 fixes

2.1 The weekly rhythm (same for all three members)

When	What to do
Monday	Sync with Anas's repo (get last week's merged code) and start a new branch for the week.
Mon – Fri	Do the week's step(s). Commit + push every time a small piece works (at least every 2 days).
Saturday	Test your step(s) with the "Test it" boxes. Fix bugs.
Sunday before 9:00 pm	Write 3 lines in <code>docs/progress/anas.md</code> , push, and open the Pull Request .
Sunday 9:30 pm	15-minute group call: what is done, what is stuck, next week's steps.
Sunday night / Monday	You merge all Pull Requests (yours first), build, play for 5 minutes, then tell the group "Week N merged — sync now".

2.2 Group milestones

Milestone	Date	What must work (whole group)
-----------	------	------------------------------

Milestone	Date	What must work (whole group)
M1	25 Oct	Program runs; menu appears; submarine moves on screen with no flicker; random numbers work
M2	8 Nov	Core game: torpedoes, falling charges, hits, explosions, score, lives, status bar
M3	18 Nov	Everything: sonar + hidden mines, sounds, levels, pause, high-score file, game-over screen
M4	22 Nov	All tests passed on a lab PC; final report written; ready for the 23 Nov demo

Tips

- **Behind schedule?** Say it in the Sunday call — don't hide it. Push only code that **builds**: an unfinished procedure keeps its STUB line, so the game still runs.
- Your plan is a little **ahead** of the proposal's timeline on purpose — the extra days are your safety buffer.

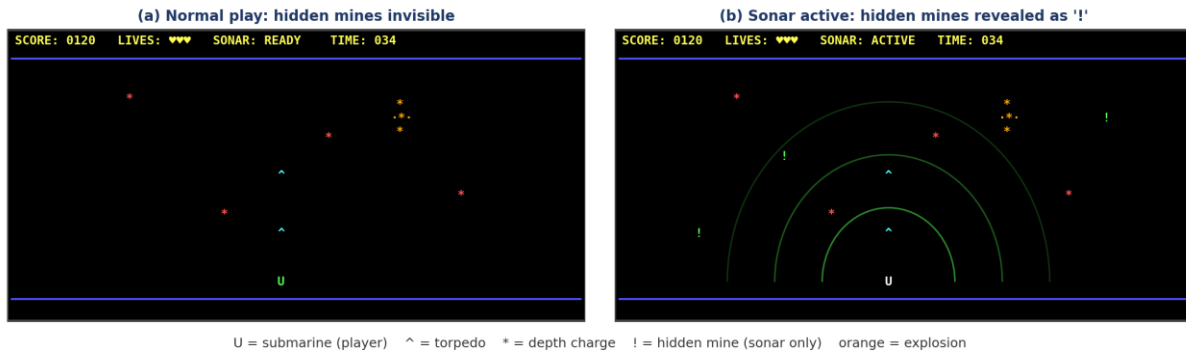
3. How the Game Works (5-minute version)

The whole game is **one loop** that repeats about **12 times per second** (every 80 ms). One repeat is called a **frame**. In every frame the program does the same five jobs, always in this order:

Job in every frame	What happens	Procedures	Who
1. Read keys	Check the keyboard without waiting ; move sub, fire, sonar, pause	<code>HandleInput</code>	Anas (you)
2. Move things	Torpedoes go up, charges fall down, new charges appear, explosions age	<code>UpdateTorpedoes</code> <code>UpdateCharges</code> , <code>SpawnCharge</code> <code>UpdateExplosions</code>	Anas (you) Ali Alyan
3. Check hits	Torpedo hits charge → points; charge hits sub → lose a life	<code>CheckCollisions</code> → <code>AddScore</code> , <code>LoseLife</code>	Anas (you) → Ali
4. Draw	Draw everything into a memory copy of the screen, then show only what changed	<code>BuildFrame</code> , <code>DrawHUD</code> , <code>Render</code>	Alyan
5. Wait	Wait the rest of the 80 ms so the speed is the same on every PC	<code>FrameDelay</code>	Anas (you)

Around the loop there are screens (menu, instructions, game over) and a **state** variable `gameState` that tells the loop what to do:

<code>gameState</code>	Meaning	What the loop does
ST_MENU (0)	Menu is showing	Loop stops, menu appears (ESC goes here)
ST_PLAYING (1)	Normal play	All five jobs
ST_SONAR (2)	Sonar is on (SPACE)	Nothing moves for 2 s; hidden mines shown as green 'I'
ST_PAUSED (3)	Paused (P)	Nothing moves; "PAUSED" text is drawn
ST_GAMEOVER (4)	No lives left	Loop stops, game-over screen appears
ST_EXIT (5)	Exit chosen in menu	Program ends



What the player sees: an 80×25 text screen (status bar on top, submarine 'U' at the bottom).

3.1 Project files and who owns them

There is **one** `.asm` file (`src/game.asm`). It **INCLUDEs** one `.inc` file per module. Every member has **one folder** and edits only that folder, so three people can work at the same time without conflicts. The full folder tree and the Git rules are in **section 5**.

Folder / file	What is inside	Owner
<code>src/game.asm</code>	<code>main</code> , <code>InitGame</code> , <code>GameLoop</code> , <code>FrameDelay</code> , all INCLUDE lines	Anas ← you
<code>src/shared/constants.inc</code> , <code>data.inc</code>	Shared numbers and variables (everyone agrees first)	Anas (all agree) ← you
<code>src/anas/</code>	<code>input.inc</code> , <code>torpedo.inc</code> , <code>collision.inc</code> , <code>stubs_anas.inc</code>	Anas ← you
<code>src/alyan/</code>	<code>render.inc</code> , <code>hud.inc</code> , <code>explosion.inc</code> , <code>sonar.inc</code> , <code>stubs_alyan.inc</code>	Alyan
<code>src/ali/</code>	<code>random.inc</code> , <code>screens.inc</code> , <code>charges.inc</code> , <code>sound.inc</code> , <code>score.inc</code> , <code>highscore.inc</code> , <code>stubs_ali.inc</code>	Ali
<code>docs/progress/<name>.md</code> <code>docs/report/<name>.md</code>	Your weekly log and your final-report section	Each member

3.2 The one object type everybody uses

The submarine is just a column number (`subX`). Every torpedo, charge and explosion is a 4-byte `GameObject` stored in an array. **active** = **0** means the slot is free. **hidden** = **1** means the charge is an invisible mine (it still moves and still hits — it is only *not drawn*).

```

data.inc (the comment shows who WRITES each variable; anyone may read it)
;=====
; data.inc - shared data (who WRITES each variable is in the comment)
;=====
GameObject STRUCT
    xPos    BYTE 0        ; column 0-79
    yPos    BYTE 0        ; row 0-24
    active   BYTE 0        ; 1 = slot in use
    hidden   BYTE 0        ; 1 = drawn only during sonar
GameObject ENDS

.data
gameState   BYTE  ST_MENU      ; Anas
subX        BYTE  40           ; Anas
score       DWORD 0           ; Ali
highScore   DWORD 0           ; Ali
lives       BYTE  3           ; Ali
level       BYTE  1           ; Ali
fireCooldown BYTE 0           ; Anas
lastFrame   DWORD 0           ; Anas
frameCount  DWORD 0           ; Anas
fallDelay   BYTE  6           ; Ali (set per level)
spawnDelay  BYTE  14          ; Ali (set per level)
fallTimer   BYTE  6           ; Ali
spawnTimer  BYTE  14          ; Ali
sonarStart  DWORD 0           ; Alyan
sonarReadyAt DWORD 0         ; Alyan
sonarPlayed BYTE 0           ; Alyan

Torpedoes    GameObject MAX_TORP DUP(<>) ; Anas
Charges      GameObject MAX_CHARGE DUP(<>) ; Ali
Explosions   GameObject MAX_EXPL  DUP(<>) ; Alyan
ExplTimer    BYTE MAX_EXPL DUP(0) ; Alyan

ScreenBuf    WORD SCREEN_W*SCREEN_H DUP(0720h) ; Alyan
PrevBuf      WORD SCREEN_W*SCREEN_H DUP(0FFFFh) ; Alyan

```

4. What You Need to Know First

Study these in **Week 0 (5 – 11 Oct)** and keep going in Week 1. The chapter numbers are from Irvine, *Assembly Language for x86 Processors*, 8th ed. Each practice program takes about 30–60 minutes and is a tiny piece of the real game.

Topic	Simple explanation	Irvine	Practice
Program basics, BYTE / DWORD, MOV, ADD, INC	How data is stored and moved	Ch 3–4	Declare <code>score</code> DWORD, <code>lives</code> BYTE 3; view them in the Memory window
CMP + jumps, unsigned JA/JB/JAE/JBE	"If" statements in assembly. Columns and times are never negative, so use unsigned jumps	Ch 6	Print "BIG" if a number is above 50
Procedures, CALL/RET, PUSH/POP, USES	Each module is a procedure. USES saves registers so a call can't destroy your loop counter	Ch 5	Write a procedure that prints a line at a position with <code>Gotoxy</code>
Arrays with a pointer, STRUCT	Walk an array: ESI points to an element, add esi, TYPE GameObject moves to the next	Ch 4, 10	Array of 5 structs; print every x, y
ReadKey (no waiting)	ZF = 1 means no key. AL = character, or AL = 0 and DX = key code for arrow keys	Ch 5	Move a letter left/right with A/D while a counter keeps running

Topic	Simple explanation	Irvine	Practice
<code>GetMseconds</code> , <code>Delay</code>	Clock in milliseconds and "sleep for EAX ms"	Ch 5	Print a dot exactly every 500 ms
Nested loops	A loop inside a loop. Use different counters (ECX outside, EDX inside)	Ch 4, 6	Print a 5×5 grid of '*'
State machine	One variable decides which part of the code runs	Ch 6	Menu: 1 = "hello", 2 = "bye", 3 = exit
<code>REP STOSB</code>	Fill many bytes with AL in one instruction (EDI = where, ECX = how many)	Ch 9	Clear a 20-byte array to 0
Git basics	clone, pull, commit, push, fix a merge conflict	—	Make a test repo with a partner and merge two changes

5. GitHub: Folder Structure and How the Team Pushes Code

5.1 The folder structure (already created by Anas)

Repository: <https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection>. Every member has **one folder** in `src/`. Because nobody edits another person's files, Git can always merge the three parts **without conflicts**.

folder structure					
Submarine-Defense-Game-with-Sonar-Detection/					
├──	README.md	.gitignore	.gitattributes	.github/CODEOWNERS	Anas
├──	Project.sln	Project.vcxproj	Project.vcxproj.filters		Anas (others NEVER commit)
├──	src/				
│	├──	game.asm		Anas	the only file that is assembled
│	├──	shared/			
│	│	├──	constants.inc	Anas	(all three agree first)
│	│	└──	data.inc	Anas	(all three agree first)
│	├──	anas/			
│	│	├──	input.inc	torpedo.inc	collision.inc
│	│	└──	stubs_anas.inc		
│	├──	alyan/			
│	│	├──	render.inc	hud.inc	explosion.inc
│	│	└──	stubs_alyan.inc		
│	├──	ali/			
│	│	├──	random.inc	screens.inc	charges.inc
│	│	├──	sound.inc	score.inc	highscore.inc
│	│	└──	stubs_ali.inc		
└──	docs/				
│	├──	proposal/	guides/		Anas
│	├──	progress/	anas.md	alyan.md	ali.md
│	├──	report/	anas.md	alyan.md	ali.md
					each member: own weekly log
					each member: own report section

	Files
You may change	<code>src/game.asm</code> , <code>src/shared/</code> , <code>src/anas/</code> , <code>docs/</code> (all), README, .gitignore, VS project files
Never change	<code>src/alyan/</code> and <code>src/ali/</code> — ask the owner to fix their own code (except in Week 6 bug-fixing, after telling them)

Rules that prevent conflicts

- Never use `git add .` — add only your own paths (shown in the commands below).
- Your module's private variables and tables go in a `.data` section **inside your own .inc file** (e.g. `FallDelayTable` lives in `score.inc`). Only truly shared variables are in `data.inc`.
- When a member asks for a shared constant/variable, add it in your weekly branch and announce it in the group.

- Never add files to the Visual Studio project — it changes `Project.vcxproj`. All files already exist.
- Testing something in `game.asm` on your PC is fine — run `git restore src/game.asm` before you commit.
- Everyone installs Irvine in `C:\Irvine`, so the same project settings work on all three laptops.

5.2 One-time setup of the repository (Week 0 — by Wed 7 Oct)

You get a ready-made **starter folder** (`Submarine-Defense-Game-starter.zip`) with the exact structure, the shared files, the stub files, empty module files with a header (owner, step, procedures), `.gitignore`, `.gitattributes`, `CODEOWNERS`, `README` and the docs folders. It was test-assembled: it builds with 0 errors, and so does the finished game in the same structure.

1. Clone your repo and copy everything from the starter folder into it (including the hidden `.gitignore`, `.gitattributes` and `.github` folder).
2. From Irvine's Visual Studio template project (the one used in the lab, e.g. `Project32_VS2022`) copy `Project.sln`, `Project.vcxproj` and `Project.vcxproj.filters` into the repo's top folder.
3. Open `Project.sln`. In Solution Explorer remove the template's sample `.asm`. **Add → Existing Item → `src\game.asm`**. Then add every `.inc` file too (so members can open them in VS). Right-click a `.inc` → Properties: *Item Type* must be **Does not participate in build**; `game.asm` must be **Microsoft Macro Assembler**.
4. Project → Properties → *All Configurations* → **Microsoft Macro Assembler** → **General** → **Include Paths** = `C:\Irvine;$(ProjectDir)src`. (Linker settings stay as in the template.)
5. Build + run: it must print *"Depth Charge skeleton builds and runs OK"*.
6. Push to `main` — this is the **only** time anyone pushes straight to `main`:

Git Bash — one time

```
git clone https://github.com/muhammadanas20/Submarine-Defense-Game-with-Sonar-Detection.git
cd Submarine-Defense-Game-with-Sonar-Detection
# ... copy the starter files + Project.* here, build once in VS ...
git status          # Debug/, .vs/ and *.user must NOT be listed (.gitignore)
git add .
git commit -m "Week 0: project structure, shared files, stubs"
git push origin main
```

7. Check that Alyan and Ali accepted the collaborator invitation, forked, cloned and built (by **Fri 9 Oct**).
8. Optional: put their usernames in `.github/CODEOWNERS`; in *Settings* → *Branches* add a rule for `main`: **Require a pull request before merging** (if your GitHub plan offers it).
9. Merge their **practice PRs** (only their `docs/progress/*.md`). Now everyone knows the process.

5.3 Your own weekly work

You work in the original repo (no fork), but still use a **weekly branch + Pull Request**, so the history stays clean and you can undo a bad week with one click.

```

Git Bash — weekly
# MONDAY
git checkout main
git pull origin main
git checkout -b week1-anas

# EVERY TIME SOMETHING SMALL WORKS
git status
git add src/game.asm src/shared src/anas docs
git commit -m "A2: main + GameLoop state machine"
git push -u origin week1-anas      # later: just git push

```

5.4 Sunday night: merge the team's Pull Requests

1. Merge **your own** PR first (*week1-anas* → *main*).
2. Open each member's PR. **Files changed** tab: only their folder *src/alyan/* or *src/ali/* and their own docs files. Anything else → *Review changes* → *Request changes* and explain.
3. Click **Merge pull request** → **Confirm merge**. Order: Anas → Alyan → Ali.
4. On your PC: `git checkout main` and `git pull origin main`. Build in VS, play for 5 minutes.
5. Post in the group: "*Week N merged — sync now*". At milestones also tag the version:

```

Git Bash — milestone tag
git tag M1          # M2, M3, and v1.0 on Sun 22 Nov
git push origin M1

```

In Week 6, after M3, delete the three stub files' `INCLUDE` lines in `game.asm` (the files should be empty by then).

5.5 If something goes wrong

Problem	Fix
<code>git status</code> lists <code>src/game.asm</code> (you tested something)	<code>git restore src/game.asm</code>
<code>Project.vcxproj / .sln / .filters</code> changed	<code>git restore Project.sln Project.vcxproj Project.vcxproj.filters</code>
Your own repo: you committed a wrong file	<code>git checkout origin/main -- <file></code> then commit and push
Conflict while merging <code>upstream/main</code>	Can only happen in your own files. Open the file, keep the right lines, delete the <<<<<<, =====, >>>>>> lines, then <code>git add <file></code> and <code>git commit</code> . Unsure? Ask Anas before pushing.
A member's PR shows "conflicts"	Ask the member to sync (<code>git fetch upstream</code> , <code>git merge upstream/main</code>) and push again — the PR updates by itself.
The game is broken after a merge	Open the merged PR on GitHub → Revert button → merge the revert PR. Then ask the owner to fix it in next week's PR.
Push rejected ("fetch first")	<code>git pull origin <your-branch></code> then <code>git push</code>
Build error A2005 <i>symbol redefinition</i>	You wrote a procedure but its STUB line is still there — delete the STUB line.
Build error A2006 <i>undefined symbol</i>	The procedure has no real version and no STUB line — tell its owner / Anas.
Which branch am I on?	<code>git branch</code> (the one with *)

6. Your Steps

Do the steps in order. Each step tells you **which file**, **what it does in simple words**, **the logic step by step**, an example, how to **test** it and the usual **mistakes**.

A1 Create the GitHub repo structure and shared files (Week 0 · 5 – 7 Oct)

File	the GitHub repo: src/src/game.asm, src/shared/constants.inc, src/shared/data.inc, src/anas/stubs_anas.inc / src/alyan/stubs_alyan.inc / src/ali/stubs_ali.inc, docs/
You write	the project skeleton that everybody forks

In simple words

- You build the "empty house" with all rooms (folders and files) in it. Everyone then fills only their own rooms.
- The starter folder already contains everything — you mainly add Irvine's VS project files and push.

The logic, step by step

1. Follow **section 5.2** exactly: copy the starter folder into your repo, add Irvine's `Project.sln/.vcxproj`, set the include path, build, push.
2. Sit with Alyan and Ali for 30 minutes and read `constants.inc` and `data.inc` together. Agree on every value. Rule: **nobody changes these two files except you**.
3. Each member's `stubs_<name>.inc` already lists the procedures other people call. `game.asm` INCLUDEs the stub files before the real modules (see below).
4. Check that both members forked, cloned and built it, and merge their practice PRs (by Fri 9 Oct).
5. Set the console to at least 80×25 (cmd → Properties → Layout).

```
src/shared/constants.inc

;=====
; constants.inc - shared constants (ALL members agree on these)
;=====
; ---- screen layout ----
SCREEN_W    = 80
SCREEN_H    = 25
HUD_ROW     = 0           ; status bar
TOP_ROW     = 2           ; first playfield row
SUB_ROW     = 23          ; submarine row
BOTTOM_ROW  = 24          ; bottom border
MIN_COL     = 1
MAX_COL     = 78

; ---- object limits ----
MAX_TORP    = 8
MAX_CHARGE  = 15
MAX_EXPL    = 5
EXPL_FRAMES = 5

; ---- timing (milliseconds) ----
FRAME_MS    = 80
SONAR_MS    = 2000
SONAR_COOLDOWN_MS = 10000

; ---- game states ----
ST_MENU     = 0
ST_PLAYING  = 1
ST_SONAR    = 2
```

```

ST_PAUSED      = 3
ST_GAMEOVER    = 4
ST_EXIT        = 5

; ---- characters ----
CH_SUB         = 'U'
CH_TORP        = '^'
CH_CHARGE      = '*'
CH_MINE        = '!'
CH_BORDER      = '='

; ---- colours: foreground + background*16 ----
ATTR_TEXT      = lightGray + (black * 16)
ATTR_SUB       = lightGreen + (black * 16)
ATTR_TORP      = lightCyan + (black * 16)
ATTR_CHARGE    = lightRed + (black * 16)
ATTR_MINE      = lightGreen + (black * 16)
ATTR_HUD       = yellow + (black * 16)
ATTR_BORDER    = lightBlue + (black * 16)

```

src/game.asm — the INCLUDE lines at the end of the file

```

; ---- temporary stub files (delete each one's INCLUDE line when the file is empty) ----
INCLUDE anas\stubs_anas.inc
INCLUDE alyan\stubs_alyan.inc
INCLUDE ali\stubs_ali.inc

; ---- Anas ----
INCLUDE anas\input.inc
INCLUDE anas\torpedo.inc
INCLUDE anas\collision.inc
; ---- Alyan ----
INCLUDE alyan\render.inc
INCLUDE alyan\hud.inc
INCLUDE alyan\explosion.inc
INCLUDE alyan\sonar.inc
; ---- Ali ----
INCLUDE ali\random.inc
INCLUDE ali\screens.inc
INCLUDE ali\charges.inc
INCLUDE ali\sound.inc
INCLUDE ali\score.inc
INCLUDE ali\highscore.inc

END main

```

src/anas/stubs_anas.inc (your own stubs, so GameLoop builds in Week 1)

```

; stubs_anas.inc - EMPTY versions of Anas's procedures.
; Anas deletes ONE line as soon as the real procedure is written.
STUB MACRO procName          ; makes:  procName PROC / ret / procName ENDP
procName PROC
    ret
procName ENDP
ENDM

.code
STUB HandleInput             ; called by GameLoop
STUB FireTorpedo             ; called by HandleInput
STUB UpdateTorpedoes         ; called by GameLoop
STUB CheckCollisions         ; called by GameLoop

```

Stubs in the other files — Alyan: InitSystem, InvalidateScreen, BuildFrame, Render, UpdateExplosions, StartExplosion, StartSonar, UpdateSonar, DrawHUD, DrawExplosions. **Ali:** SeedRandom, ShowMenu, GameOverScreen, LoadHighScore, SpawnCharge, UpdateCharges, AddScore,

LoseLife, ApplyLevel, PlayFireSound, PlayExplosionSound, PlaySonarSweep, PlayLifeLostSound, PlayGameOverSound, SaveHighScore.

Test it — the step is DONE when...

- The project builds with 0 errors on all three laptops and prints the "skeleton OK" message.
- Both members' practice Pull Requests are merged.

A2 Main program and game loop (Week 1 · 12 – 15 Oct)

File	src/game.asm
You write	main, GameLoop
You call	InitSystem, LoadHighScore, ShowMenu, InitGame, all update/draw procedures

In simple words

- `main` is the outer loop: `menu` → **play one game** → **back to menu**.
- `GameLoop` is the inner loop: **one pass = one frame**. It looks at `gameState` and decides which jobs to do.

The logic, step by step

1. `main`: call `InitSystem` and `LoadHighScore` once. Then repeat: `ShowMenu`; if `gameState = ST_EXIT` quit; else `InitGame` and `GameLoop`.
2. `GameLoop`: first `HandleInput`. Then load `gameState` into `AL` (it is a BYTE) and compare:
3. `PLAYING` → update everything + check collisions, then draw. `SONAR` → only `UpdateSonar`, then draw. `PAUSED` → only draw. Anything else (`GAMEOVER` / `MENU`) → leave the loop.
4. Drawing is always: `BuildFrame` → `Render` → `FrameDelay`, then `jmp NextFrame`.
5. When the loop ends with `ST_GAMEOVER`, call Ali's `GameOverScreen` before returning.
6. For now write a **short** `InitGame` with only `mov gameState, ST_PLAYING` and `GetMseconds` → `lastFrame`. You complete it in A7.

```
game.asm - main
main PROC
    call InitSystem           ; Alyan: seed random, hide cursor
    call LoadHighScore        ; Ali  : read highscore.dat
MenuLoop:
    call ShowMenu             ; Ali  : sets gameState = PLAYING or EXIT
    cmp  gameState, ST_EXIT
    je   Quit
    call InitGame
    call GameLoop             ; returns when the game is over
    jmp  MenuLoop
Quit:
    call Clrscr
    exit
main ENDP
```

```
game.asm - GameLoop
;-----
; GameLoop: one pass = one frame. Returns on GAME OVER / ESC.
;-----
GameLoop PROC
NextFrame:
    call HandleInput
    mov  al, gameState
    cmp  al, ST_PLAYING
    je   DoPlaying
```

```

    cmp    al, ST_SONAR
    je     DoSonar
    cmp    al, ST_PAUSED
    je     DoDraw          ; paused: draw only, no updates
    jmp    GameEnded       ; ST_GAMEOVER or ST_MENU

DoPlaying:
    call   UpdateTorpedoes  ; Anas
    call   UpdateCharges    ; Ali
    call   SpawnCharge      ; Ali
    call   UpdateExplosions  ; Alyan
    call   CheckCollisions  ; Anas (may set ST_GAMEOVER)
    jmp    DoDraw

DoSonar:
    call   UpdateSonar      ; Alyan (may set ST_PLAYING)

DoDraw:
    call   BuildFrame       ; Alyan
    call   Render           ; Alyan
    call   FrameDelay       ; Anas
    jmp    NextFrame

GameEnded:
    cmp    gameState, ST_GAMEOVER
    jne    BackToMenu
    call   GameOverScreen   ; Ali (calls SaveHighScore)
BackToMenu:
    ret
GameLoop ENDP

```

Delete the day-one test `main` from the starter. Until Alyan's renderer exists you can't see anything. To test early, put a temporary `mov dh, 23 / mov dl, subX / call Gotoxy / mov al, 'U' / call WriteChar` before `FrameDelay`, and remove it at M1.

Test it — the step is DONE when...

- The program builds with the stubs and runs without crashing.
- In the debugger (F10) you can see the loop going round; setting `gameState` to 4 in the Watch window makes it return.

Common mistakes

- Forgetting `jmp NextFrame` — the loop runs only once.
- `cmp gameState, eax` — size mismatch. `gameState` is a BYTE: use `AL`.

A3 Frame timing (Week 1 · 16 – 18 Oct)

File	src/game.asm
You write	<code>FrameDelay</code>
You call	<code>GetMseconds, Delay</code> (Irvine32)
Called by	GameLoop (end of every frame)

In simple words

- Every frame must take **80 ms**. If drawing took 20 ms, we only sleep the other 60 ms.
- So the game runs at the same speed on a fast laptop and a slow lab PC.

The logic, step by step

1. `elapsed = now - lastFrame`.

2. If `elapsed < FRAME_MS`, call `Delay` with `(FRAME_MS - elapsed)`.
3. Save the new time in `lastFrame` and add 1 to `frameCount`.

```
game.asm - FrameDelay
;-----
; FrameDelay: waits until FRAME_MS have passed since lastFrame
;-----
FrameDelay PROC USES eax
    call GetMseconds
    sub  eax, lastFrame           ; eax = elapsed ms this frame
    cmp  eax, FRAME_MS
    jae  NoWait                  ; frame was already slow
    neg  eax
    add  eax, FRAME_MS           ; eax = FRAME_MS - elapsed
    call Delay                   ; Irvine32: sleep EAX ms
NoWait:
    call GetMseconds
    mov  lastFrame, eax
    inc  frameCount
    ret
FrameDelay ENDP
```

Test it — the step is DONE when...

- `frameCount` grows by about 12 every second (watch it in the debugger).
- The game speed feels the same on two different PCs.

Common mistakes

- Just `Delay 80` every frame — the game becomes slower on slow PCs.
- An empty counting loop (`mov ecx, 1000000 / loop`) for waiting — speed depends on the CPU and uses 100% CPU.

A4 Keyboard input and submarine movement (Week 2 · 19 – 25 Oct)

File	src/anas/input.inc
You write	<code>HandleInput</code>
You call	<code>ReadKey</code> , <code>FireTorpedo</code> , <code>StartSonar</code> (Alyan)
Called by	<code>GameLoop</code> (start of every frame)

In simple words

- Check the keyboard, but **never wait** for a key — the game must keep moving.
- A/D or arrows move the sub one column. W fires. SPACE = sonar. P = pause. ESC = back to menu.

The logic, step by step

1. Call `ReadKey`. If **ZF = 1**, there is no key → return.
2. If `AL = 0` it is an arrow key: compare **DX** with `VK_LEFT` / `VK_RIGHT`.
3. Check ESC (1Bh) and SPACE **first**, then do `or al, 20h` (turns 'A' into 'a') and compare with 'a', 'd', 'w', 'p'.
4. Move / fire / sonar only when `gameState = ST_PLAYING`.
5. Move left only if `subX > MIN_COL`; right only if `subX < MAX_COL` (unsigned jumps).
6. SPACE: only if the time now $\geq$ `sonarReadyAt` (cool-down finished), then call `StartSonar`.
7. Jump back to `ReadKey` until no key is left, so a held key never lags.

input.inc - HandleInput

```

;-----
; HandleInput: non-blocking. Called once per frame. Processes
; every key waiting in the buffer so held keys never "lag".
;-----
HandleInput PROC USES eax edx
NextKey:
    call ReadKey                ; ZF = 1 -> no key waiting
    jz   Done

    cmp  al, 0                  ; AL = 0 -> special key (arrows)
    je   Extended
    cmp  al, 1Bh                ; ESC
    je   KeyEsc
    cmp  al, ' '
    je   KeySpace
    or   al, 20h                ; letters -> lower case
    cmp  al, 'a'
    je   KeyLeft
    cmp  al, 'd'
    je   KeyRight
    cmp  al, 'w'
    je   KeyFire
    cmp  al, 'p'
    je   KeyPause
    jmp  NextKey

Extended:
    cmp  dx, VK_LEFT           ; DX = virtual-key code
    je   KeyLeft
    cmp  dx, VK_RIGHT
    je   KeyRight
    jmp  NextKey

KeyLeft:
    cmp  gameState, ST_PLAYING
    jne  NextKey
    cmp  subX, MIN_COL         ; boundary check
    jbe  NextKey
    dec  subX
    jmp  NextKey

KeyRight:
    cmp  gameState, ST_PLAYING
    jne  NextKey
    cmp  subX, MAX_COL
    jae  NextKey
    inc  subX
    jmp  NextKey

KeyFire:
    cmp  gameState, ST_PLAYING
    jne  NextKey
    call FireTorpedo
    jmp  NextKey

KeySpace:
    cmp  gameState, ST_PLAYING
    jne  NextKey
    call GetMseconds
    cmp  eax, sonarReadyAt     ; still cooling down?
    jb   NextKey
    call StartSonar            ; Alyan
    jmp  NextKey

KeyPause:
    cmp  gameState, ST_PLAYING
    je   GoPause
    cmp  gameState, ST_PAUSED
    jne  NextKey
    mov  gameState, ST_PLAYING
    jmp  NextKey

```

```
GoPause:
    mov     gameState, ST_PAUSED
    jmp     NextKey
KeyEsc:
    mov     gameState, ST_MENU      ; GameLoop returns to the menu
Done:
    ret
HandleInput ENDP
```

Test it — the step is DONE when...

- A/D and ←/→ move the 'U' smoothly and it stops at the borders (columns 1 and 78).
- Caps Lock on still works. Releasing the key stops the sub at once.
- **Milestone M1 (25 Oct):** with Alyan's renderer and Ali's menu, the sub moves on screen with no flicker.

Common mistakes

- Doing `or al, 20h` before checking ESC/SPACE/0: 0 turns into SPACE and ESC into ';'.
 • Using `ReadChar` — it waits for a key and freezes the game.

A5 Torpedoes (Week 3 · 26 Oct – 1 Nov)

File	src/anas/torpedo.inc
You write	<code>FireTorpedo</code> , <code>UpdateTorpedoes</code>
You call	<code>PlayFireSound</code> (Ali)
Called by	<code>HandleInput</code> (W key), <code>GameLoop</code>

In simple words

- There are 8 torpedo "slots". Firing = find a free slot and put a torpedo just above the sub.
- Every frame each active torpedo goes up one row. At the top it is removed (slot becomes free).

The logic, step by step

1. `FireTorpedo`: if `fireCooldown > 0`, do nothing (stops the player firing a solid wall).
2. Loop over `Torpedoes` with ESI. First slot with `active = 0` → set `xPos = subX`, `yPos = SUB_ROW - 1`, `active = 1`.
3. Set `fireCooldown = 3` (3 frames between shots) and call `PlayFireSound`.
4. `UpdateTorpedoes`: decrease `fireCooldown` if `> 0`. For each active torpedo: `dec yPos`; if `yPos < TOP_ROW`, set `active = 0`.
5. Move to the next element with `add esi, TYPE GameObject` (4 bytes).

```
torpedo.inc
;-----
; FireTorpedo: puts a torpedo in the first free slot
;-----
FireTorpedo PROC USES eax ecx esi
    cmp     fireCooldown, 0
    ja      Done
    mov     esi, OFFSET Torpedoes
    mov     ecx, MAX_TORP
FindSlot:
    cmp     (GameObject PTR [esi]).active, 0
    je      Found
    add     esi, TYPE GameObject      ; next element (4 bytes)
    loop    FindSlot
    jmp     Done                      ; all 8 torpedoes in flight
```

```

Found:
    mov     al, subX
    mov     (GameObject PTR [esi]).xPos, al
    mov     (GameObject PTR [esi]).yPos, SUB_ROW - 1
    mov     (GameObject PTR [esi]).active, 1
    mov     fireCooldown, 3           ; 3 frames between shots
    call    PlayFireSound             ; Ali
Done:
    ret
FireTorpedo ENDP

;-----
; UpdateTorpedoes: moves every active torpedo up one row
;-----
UpdateTorpedoes PROC USES ecx esi
    cmp     fireCooldown, 0
    je      @F
    dec     fireCooldown
@@:
    mov     esi, OFFSET Torpedoes
    mov     ecx, MAX_TORP
L1:
    cmp     (GameObject PTR [esi]).active, 0
    je      NextT
    dec     (GameObject PTR [esi]).yPos
    cmp     (GameObject PTR [esi]).yPos, TOP_ROW
    jae     NextT
    mov     (GameObject PTR [esi]).active, 0 ; left the playfield
NextT:
    add     esi, TYPE GameObject
    loop    L1
    ret
UpdateTorpedoes ENDP

```

Test it — the step is DONE when...

- Holding W fires a stream with small gaps, never more than 8 torpedoes.
- Torpedoes vanish at the top border and their slots are reused.

Common mistakes

- add `esi, 1` instead of `add esi, TYPE GameObject`.
- Moving torpedoes without checking `active` first.

A6 Collision detection (Week 4 · 2 – 8 Nov)

File	src/anas/collision.inc
You write	<code>CheckCollisions</code>
You call	<code>AddScore</code> , <code>LoseLife</code> , <code>PlayExplosionSound (Ali)</code> , <code>StartExplosion (Alyan)</code>
Called by	GameLoop (after all moves)

In simple words

- Compare **every torpedo with every charge**. Same column and same row → hit.
- Also compare every charge with the submarine. If a charge reaches the sub's row in the sub's column → lose a life.
- We check the **arrays**, not the screen — that is why invisible mines still hit.

The logic, step by step

1. Outer loop: torpedoes (ESI, counter ECX with **LOOP**). Inner loop: charges (EDI, counter **EDX** with **DEC/JNZ**). Skip inactive ones.
2. Hit if same column **and** (same row **or** the torpedo is exactly one row above the charge). The second case catches a torpedo and a charge that **jumped over each other** in the same frame.
3. On a hit: set both **active = 0**; **EAX = 10** (or 25 if **hidden = 1**) → **AddScore**; **AL = col**, **AH = row** → **StartExplosion**; **PlayExplosionSound**; then go to the **next torpedo** (this one is used up).
4. Second loop: every active charge with **yPos = SUB_ROW** and **xPos = subX** → remove it, **StartExplosion**, **LoseLife**. Stop if the state became **ST_GAMEOVER**.

```
collision.inc - CheckCollisions
CheckCollisions PROC USES eax ebx ecx edx esi edi
;----- 1) torpedo vs charge : nested loops -----
mov esi, OFFSET Torpedoes
mov ecx, MAX_TORP                ; outer counter (LOOP)
TorpLoop:
  cmp (GameObject PTR [esi]).active, 0
  je NextTorp
  mov edi, OFFSET Charges
  mov edx, MAX_CHARGE            ; inner counter (DEC/JNZ)
ChargeLoop:
  cmp (GameObject PTR [edi]).active, 0
  je NextCharge
  mov al, (GameObject PTR [esi]).xPos
  cmp al, (GameObject PTR [edi]).xPos
  jne NextCharge                ; different column
  mov al, (GameObject PTR [esi]).yPos ; torpedo row
  mov bl, (GameObject PTR [edi]).yPos ; charge row
  cmp al, bl
  je Hit                        ; same cell
  inc al
  cmp al, bl                    ; torpedo one row ABOVE charge
  jne NextCharge                ; -> they crossed this frame
Hit:
  mov (GameObject PTR [esi]).active, 0
  mov (GameObject PTR [edi]).active, 0
  mov eax, 10                    ; visible charge = 10 points
  cmp (GameObject PTR [edi]).hidden, 0
  je @F
  mov eax, 25                    ; hidden mine = 25 points
@@: call AddScore
  mov al, (GameObject PTR [edi]).xPos
  mov ah, (GameObject PTR [edi]).yPos
  call StartExplosion            ; Alyan: AL = col, AH = row
  call PlayExplosionSound        ; Ali
  jmp NextTorp                  ; this torpedo is gone
NextCharge:
  add edi, TYPE GameObject
  dec edx
  jnz ChargeLoop
NextTorp:
  add esi, TYPE GameObject
  loop TorpLoop

;----- 2) charge vs submarine -----
mov edi, OFFSET Charges
mov ecx, MAX_CHARGE
SubLoop:
  cmp (GameObject PTR [edi]).active, 0
  je NextS
  cmp (GameObject PTR [edi]).yPos, SUB_ROW
```

```

jne NextS
mov al, (GameObject PTR [edi]).xPos
cmp al, subX
jne NextS
mov (GameObject PTR [edi]).active, 0
mov ah, SUB_ROW
call StartExplosion
call LoseLife
cmp gameState, ST_GAMEOVER
je Done
NextS:
    add edi, TYPE GameObject
    loop SubLoop
Done:
    ret
CheckCollisions ENDP

```

Test it — the step is DONE when...

- A torpedo fired under a charge always destroys it — also at high levels when charges are fast.
- An invisible mine gives 25 points when hit and can still destroy the sub.
- **Milestone M2 (8 Nov):** fire, hit, score, explosions and lives all work together.

Common mistakes

- Using ECX for both loops — the inner loop destroys the outer counter.
- No "one row above" check — about half the shots pass through the charge.
- Forgetting `jmp NextTorp` after a hit — one torpedo destroys several charges.

A7 New-game reset and pause (Week 5 · 9 – 15 Nov)

File	src/game.asm (+ P key already in src/anas/input.inc)
You write	<code>InitGame</code>
You call	<code>ApplyLevel</code> (Ali), <code>InvalidateScreen</code> (Alyan)
Called by	main (before every game)

In simple words

- Before every new game, put **everything back to the start**: score 0, 3 lives, level 1, sub in the middle, no objects.
- Pause: P switches between PLAYING and PAUSED. The loop already skips all moves while paused.

The logic, step by step

1. Set score, lives, level, `subX`, cool-downs. Call `ApplyLevel` and copy the delays into the two timers.
2. Clear the three object arrays with `REP STOSB` (AL = 0, EDI = array, ECX = `SIZEOF` array).
3. `Clsrscr` and `InvalidateScreen` (menus used `Clsrscr`, so the renderer must redraw everything).
4. Set `gameState` = `ST_PLAYING` and `lastFrame` = now.

game.asm — InitGame

```

;-----
; InitGame: resets every variable for a new game
;-----
InitGame PROC USES eax ecx edi
    mov score, 0
    mov lives, 3
    mov level, 1

```

```

mov  subX, 40
mov  fireCooldown, 0
mov  sonarReadyAt, 0
call ApplyLevel          ; sets fallDelay / spawnDelay
mov  al, fallDelay
mov  fallTimer, al
mov  al, spawnDelay
mov  spawnTimer, al

; clear all three object arrays with REP STOSB (AL = 0)
cld
mov  al, 0
mov  edi, OFFSET Torpedoes
mov  ecx, SIZEOF Torpedoes
rep  stosb
mov  edi, OFFSET Charges
mov  ecx, SIZEOF Charges
rep  stosb
mov  edi, OFFSET Explosions
mov  ecx, SIZEOF Explosions
rep  stosb

call Clrscr
call InvalidateScreen    ; Alyan: force a full redraw
mov  gameState, ST_PLAYING
call GetMseconds
mov  lastFrame, eax
ret
InitGame ENDP

```

Test it — the step is DONE when...

- Play → lose → menu → play again: no old charges, score 0, lives 3.
- P freezes the game and shows "PAUSED"; P again continues.

A8 Integration, testing and final report — your leader job (Week 6 · 16 – 22 Nov)

In simple words

- Join the three parts together **often** (at every milestone), not only at the end. Small merges = small problems.

Milestones for the whole group

Milestone	Date	What must work (whole group)
M1	25 Oct	Program runs; menu appears; submarine moves on screen with no flicker; random numbers work
M2	8 Nov	Core game: torpedoes, falling charges, hits, explosions, score, lives, status bar
M3	18 Nov	Everything: sonar + hidden mines, sounds, levels, pause, high-score file, game-over screen
M4	22 Nov	All tests passed on a lab PC; final report written; ready for the 23 Nov demo

Merging

Every Sunday night you merge the three Pull Requests exactly as in **section 5.4**, and tag each milestone (M1, M2, M3, v1.0). In this week: after M3 (Wed 18 Nov) remove the stub files' INCLUDE lines, then accept **only bug fixes**.

Full test checklist (run on a lab PC before 22 Nov)

✓	Test	Expected result	Owner
☐	Menu	Shows 3 options and high score; 2 = instructions; 3 / ESC = exit	Ali
☐	Movement	A/D/arrows move the sub; stops at the borders; no lag	Anas
☐	Firing	W fires; max 8 torpedoes; small gap between shots	Anas
☐	Charges	Appear at random columns; about 1 in 5 invisible	Ali
☐	Hits	Every direct hit gives +10 (+25 hidden) — also at level 9	Anas
☐	Explosion	Animated explosion for about 5 frames	Alyan
☐	Lives / game over	Lose 1 life per hit; game over after 3	Ali
☐	Levels	Level goes up every 200 points; charges get faster	Ali
☐	Sonar	SPACE: freeze 2 s, '!' marks, sweep sound; 10 s cool-down on status bar	Alyan
☐	Pause	P pauses and continues; nothing moves while paused	Anas
☐	Sounds	Different sounds for fire, explosion, sonar, life lost, game over	Ali
☐	High score	Saved and still there after restarting the program	Ali
☐	Drawing	No flicker; status bar always correct	Alyan
☐	Replay	Game over → menu → new game starts completely fresh	Anas
☐	Long run	10 minutes of play with no crash or slow-down	All

Other leader duties

- Submit the proposal on Google Classroom by **Thu 15 Oct**.
- Run the **Sunday 9:30 pm** call (15 min): what is done, what is stuck, next week's steps.
- Collect each member's report section (their modules) and put the final report together: flowchart, screenshots, code explanation, AI-use disclosure.
- Before the demo, hold a **mock viva**: each member explains another member's code.

7. Viva Preparation

Questions about your part

1. Why can't the game use `ReadChar`? How does `ReadKey` tell you that no key was pressed?
2. Explain how `FrameDelay` keeps the same speed on every PC.
3. Walk through `CheckCollisions`. Why do the two loops need different counters?
4. What is the "crossing" problem and how does the extra row check fix it?
5. Draw the state machine. Which key or event moves the game into each state?
6. Why `add esi, TYPE GameObject` and not `add esi, 1`? What is `TYPE GameObject`?
7. How does `InitGame` clear the arrays? Which registers does `REP STOSB` use?

Questions about the other members' parts (you can be asked these too)

1. How does the renderer avoid flicker? (two buffers, only changed cells are drawn) — Alyan
2. Where is row 5, column 10 in `ScreenBuf`? $((5 \times 80 + 10) \times 2 = 820)$ — Alyan
3. How does `Rand` work, and why must `EDX` be 0 before `DIV`? — Ali
4. How is the high score saved in the file? — Ali

8. Your Checklist

✓	Step	Target date
☐	A1 repo structure pushed (starter + VS project), builds on your PC	Wed 7 Oct
☐	Alyan and Ali forked, cloned, built; practice PRs merged	Sun 11 Oct
☐	Studied the topics in section 4	Sun 11 Oct
☐	Proposal submitted on Google Classroom	Thu 15 Oct
☐	A2 main + GameLoop + short InitGame, A3 FrameDelay — PR merged	Sun 18 Oct
☐	A4 HandleInput — PR merged; M1 merged and tagged	Sun 25 Oct
☐	A5 torpedoes — PR merged	Sun 1 Nov
☐	A6 CheckCollisions — PR merged; M2 merged and tagged	Sun 8 Nov
☐	A7 InitGame reset + pause — PR merged	Sun 15 Nov
☐	M3 feature freeze: all merged, stub INCLUDEs removed, tagged	Wed 18 Nov
☐	Full test checklist passed on a lab PC; final report assembled; v1.0 tagged	Sun 22 Nov

About the example code

All code comes from one reference version of the game (folder `guide/reference_code/`). It was checked with a MASM-compatible assembler (JWasm) and the Irvine32 include files — the full game, the day-one starter repo and the in-between versions with stub files all assemble with 0 errors — but it has **not** been run on Windows yet. Test every step on your own PC exactly as the "Test it" boxes say, and treat the code as a starting point that you must understand.